ctc_asr

애플리케이션 내에서 바로 동작하는 한국어 음성 인식 라이브러리 (fine-tuned Wav2Vec2-CTC), 준실시간 스트리밍 지원.

GitHub: github.com/Aditi1612/ctc_asr_library · 모델: huggingface.co/fluffy16/ctc-asr-ko (승인제) · v0.1.0

개요

한국어 ASR 모델을 **애플리케이션 프로세스 안에서 직접** 실행하는 Python 라이브러리입니다. HTTP 서버도, API 엔드포인트 호출도 필요 없습니다. 모델을 한 번만 로드한 뒤 함수를 호출하면 됩니다.

- **단건(one-shot)**: 오디오 파일이나 파형을 넘기면 전사 결과를 돌려줍니다.
- **스트리밍**: 마이크 프레임을 계속 밀어 넣으면, 화자가 말을 멈출 때마다 해당 발화의 전사 결과를 반환합니다 (음성 구간 검출 기반 분할).

디코딩 경로(오디오 정규화, 자모 정규화, 한글 조합, CTC 디코딩)는 운영 중인 API와 동일하므로 출력 결과도 같습니다.

요구 사항

구성 요소	설명
Python	3.9 이상
`torch`, `torchaudio`	사용하는 환경(CUDA 또는 CPU)에 맞는 빌드를 먼저 설치하세요. 용량이 크고 하드웨어에 종속적입니다.
GPU	선택 사항. CUDA에서는 클립당 약 25 ms, CPU에서도 충분히 빠릅니다(오디오 1초당 약 0.04 s).
디스크	모델 약 1.3 GB. 최초 1회 다운로드 후 `~/.cache/huggingface` 에 캐시됩니다.

설치

```
# 1. 하드웨어에 맞는 torch 설치 (예: CUDA 12.1)
pip install torch torchaudio --index-url https://download.pytorch.org/whl/cu121
#    ...또는 CPU 전용:
pip install torch torchaudio --index-url https://download.pytorch.org/whl/cpu

# 2. 라이브러리 설치 (공개 GitHub 저장소)
pip install "ctc-asr[hub] @ git+https://github.com/Aditi1612/ctc_asr_library.git"

# 3. Hugging Face 로그인 (모델 저장소는 승인제, "모델을 가져오는 위치" 참고)
huggingface-cli login                # HF Read 토큰 붙여 넣기, 또는: export HF_TOKEN=hf_...
```

선택적 extras:

Extra	추가 기능
`[hub]`	Hugging Face Hub에서 모델 다운로드 (권장)
`[vad]`	`webrtcvad` — 내장 에너지 기반 방식보다 정확한 음성 구간 검출
`[mic]`	`sounddevice` — 마이크 데모 실행 시에만 필요

빠른 시작 — 단건 전사

```
from ctc_asr import Transcriber

asr = Transcriber(hf_repo="fluffy16/ctc-asr-ko")   # 최초 실행 시 약 1.2 GB 다운로드, 이후 캐시

print(asr.transcribe("clip.wav"))                  # -> "카촉틀과 토이띠차 내뒤에 안자쩌요"

# 파일 대신 원본 파형(mono float32)을 직접 전달:
import soundfile as sf
wav, sr = sf.read("clip.wav", dtype="float32")
print(asr.transcribe(wav, input_sr=sr))

# 상세 결과 (원본 텍스트, reference_text 전달 시 CER, 처리 시간):
asr.transcribe("clip.wav", detailed=True)
```

입력으로 파일 경로, 인코딩된 바이트(`wav/flac/ogg` , ffmpeg가 있으면 `mp3/m4a` 도 가능), 또는 numpy / torch 파형을 받습니다.

스트리밍 (준실시간)

애플리케이션이 어디서든 오디오를 캡처해 길이에 상관없이 mono float32 청크로 밀어 넣습니다. 화자가 말을 멈추면 그때까지 버퍼링된 발화가 전사되어 반환됩니다. 부분 가설(partial)은 없으며, 발화 단위로 완성된 전사만 제공됩니다.

```
from ctc_asr import Transcriber

asr = Transcriber(hf_repo="fluffy16/ctc-asr-ko")
stream = asr.stream(on_segment=lambda s: print(f"[{s.start_time:.1f}s] {s.text}"))

for chunk in your_mic_source():                # 예: 20~100 ms 블록, 샘플레이트 무관
    for seg in stream.accept(chunk, input_sr=48000):   # 필요 시 내부에서 리샘플링
        handle(seg.text, seg.start_time, seg.end_time)

last = stream.flush()                          # 스트림 종료 시 남은 발화를 마무리 처리
```

`accept()` 는 이번 호출에서 확정된 `Segment` 목록을 반환합니다. 보통은 비어 있고, 발화가 막 끝났을 때 하나가 담깁니다. 마이크 입력을 기다리며 블로킹하지 않으며, 멈춤(pause)이 감지될 때만 모델이 실행됩니다.

`asr.stream(...)` 튜닝

인자	기본값	의미
`end_silence_ms`	600	발화를 끝내는 침묵 길이 — 체감 지연 시간과 거의 같음
`min_speech_ms`	250	이보다 짧은 발화는 잡음으로 간주해 버림
`max_speech_ms`	15000	멈춤 없이 이어지는 구간을 강제로 끊는 한계
`start_speech_frames`	3	캡처를 시작하는 데 필요한 연속 음성 프레임 수
`pre_roll_ms`	200	트리거 직전 오디오를 함께 보관해 발화 시작이 잘리지 않게 함
`vad_aggressiveness`	2	webrtcvad 민감도 0~3 (높을수록 비음성 판정이 공격적)

모델을 가져오는 위치

해석 우선순위: 명시적 경로 → Hugging Face 저장소 → 로컬 sibling 체크아웃.

소스	방법
Hugging Face Hub	`Transcriber(hf_repo="fluffy16/ctc-asr-ko")` 또는 `export CTC_HF_REPO=fluffy16/ctc-asr-ko`
로컬 파일	`Transcriber(model_dir="/path/checkpoint", vocab_path="/path/vocab.json")` 또는 `CTC_MODEL_DIR` / `CTC_VOCAB_PATH`
버전 고정	`Transcriber(hf_repo="fluffy16/ctc-asr-ko", hf_revision="<commit-sha>")`
코드에서 토큰 전달	`Transcriber(hf_repo=..., hf_token="hf_...")` (`huggingface-cli login` 대신 사용 가능)

> Hugging Face 모델 저장소는 **승인제(gated)**입니다. 누구나 저장소를 볼 수 있지만, 승인된 계정만 가중치를 다운로드할 수 있습니다.
>
> 1. huggingface.co에 로그인한 뒤 fluffy16/ctc-asr-ko를 열고 **Request access**를 클릭합니다. 관리자가 이미 계정을 추가해 둔 경우에는 이 단계를 건너뜁니다.
> 2. 승인을 기다립니다.
> 3. huggingface.co/settings/tokens에서 **Read** 토큰을 만든 뒤 `huggingface-cli login` 을 실행하거나 `HF_TOKEN` 을 설정합니다.
>
> 토큰은 비밀번호처럼 다루고 git에 커밋하지 마세요.

성능 (샘플 세트 측정값)

모델이 이미 로드된 상태에서 `transcribe()` 호출만 측정한 시간입니다.

오디오 길이	GPU (CUDA)	CPU
2 초	0.012 초	0.11 초
5 초	0.025 초	0.20 초
14 초	0.026 초	0.62 초
22 초	0.034 초	0.83 초

- **GPU**: 클립 길이와 무관하게 약 25 ms (실시간 배속 계수 RTF ≈ 0.003).
- **CPU**: 오디오 1초당 약 0.04초, 길이에 선형 비례 (RTF ≈ 0.04).
- **메모리로 모델 로드**: `Transcriber(...)` 시점에 1회, 약 0.4초.
- **Hugging Face 최초 다운로드**: 약 1.2 GB — 네트워크에 따라 다르며 이후 영구 캐시.

제약 사항

- 모델은 발화 전체를 전사합니다. 단어 단위 실시간 출력은 없으며, “실시간”은 멈춤 직후 전사가 나타난다는 의미입니다.
- 침묵을 기준으로 분할하므로 경계에서 단어가 잘릴 수 있습니다. 멈춤 없이 길게 말한 문장은 같은 오디오를 하나의 클립으로 보낼 때와 결과가 약간 다를 수 있습니다.
- 하나의 `Transcriber`는 한 번에 하나의 추론만 수행합니다. 병렬 처리가 필요하면 워커 스레드 / 프로세스마다 인스턴스를 하나씩 두세요.
- 출력은 한국어 전용이며, 모델은 마비말·비정형 발화에 맞게 파인튜닝되어 있습니다.

업데이트

시점	관리자 작업	사용자 작업
라이브러리 코드 변경	GitHub 저장소에 `git push`	`pip install --upgrade "git+https://github.com/Aditi1612/ctc_asr_library.git"`
모델 체크포인트 변경	Hugging Face 업로드 스크립트 재실행	없음 — 다음 실행 시 새 스냅샷을 받음 (`hf_revision`을 고정하지 않은 경우)

문제 해결

증상	해결
다운로드 시 `GatedRepoError` 또는 `401`	접근이 아직 승인되지 않았거나 토큰이 설정되지 않았습니다. 모델 페이지를 확인한 뒤 `huggingface-cli login` 을 실행하거나 `HF_TOKEN` 을 설정하세요.
`No checkpoint found`	`hf_repo=` / `model_dir=` 를 전달하거나 `CTC_HF_REPO` 를 설정
`Hugging Face Hub support needs 'huggingface_hub'`	`[hub]` extra로 설치
VAD 백엔드가 `energy` 이고 검출이 거침	`pip install webrtcvad`
mp3 / m4a 파일 디코딩 실패	시스템에 ffmpeg 설치
CUDA out of memory	`Transcriber(force_cpu=True)`